

ABCdeZ.jl: Simulation-based Bayesian inference and model evidence estimation in Julia

Maurice Langhinrichs¹, Thomas Höfer¹, and Nils B. Becker¹

¹ Division of Theoretical Systems Biology, German Cancer Research Center, Heidelberg 69120, Germany  Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Bayesian inference provides a statistical framework for understanding complex phenomena by identifying the most plausible explanations for the observations. Approximate Bayesian Computation (ABC) applies this framework to cases where the likelihood function is unavailable, requiring only that models can be simulated. ABCdeZ.jl is a general-purpose, simulation-based Bayesian inference package for the Julia programming language, enabling parameter estimation and model comparison using Sequential Monte Carlo (SMC) methods. In particular, ABCdeZ.jl provides model evidence estimates for each considered model individually, enabling flexible and scalable model comparison workflows in which alternative models can be added or removed without recomputing previous inferences. ABCdeZ.jl is accompanied by comprehensive documentation, an extensive test suite, and accessible example applications.

Statement of need

Approximate Bayesian Computation (ABC) methods are widely used for Bayesian parameter inference in scientific applications where likelihood functions are unavailable or computationally prohibitive. ABC is still costly but likelihood-free, being based purely on forward simulation of a generative model.

In addition to parameter inference for a given model, there is often a need for ranking alternative models. In the Bayesian setting, this is achieved by ranking the model evidences, also called marginal likelihoods. This model comparison requires efficient methods to estimate the model evidence by forward simulation. As research progresses, models of interest may be added or removed, so that the evidence should ideally be estimated separately per model, without the need for costly recomputation as the model set changes. There is currently no Julia package that provides efficient likelihood-free individual model evidence estimates.

ABCdeZ.jl was developed to address this gap while also supporting standard posterior parameter inference. In addition to model comparison and parameter inference, the package integrates features important for computationally intensive simulation-based workflows, including straightforward parallelization and data blobs that enable reuse of partial simulation results associated with posterior particles after inference.

State of the field

Several Approximate Bayesian Computation implementations exist in Julia, providing tools for parameter inference, including [ApproxBayes.jl](#), [SimulationBasedInference.jl](#), [SimulatedAnnealingABC.jl](#) and [GpABC.jl](#) (Tankhilevich et al., 2020). In some cases, model comparison is also implemented via rejection, Markov Chain Monte Carlo, or Sequential Monte

39 Carlo (SMC) methods. In particular, approaches such as those implemented in `ApproxBayes.jl`
40 or `GpABC.jl` directly estimate posterior model probabilities through model-switching Monte
41 Carlo moves, foregoing the estimation of individual normalized evidences. A similar procedure
42 is used by `pyABC` (Schälte et al., 2022), a popular ABC package in the Python programming
43 language.

44 Crucially, these implementations require all candidate models to be included within a single
45 joint inference run. As a consequence, model comparison is tied to a fixed set of models
46 and relies on parallel evaluation of all candidate models. Adding or removing models later on
47 therefore requires recomputing the full inference procedure in order to obtain posterior model
48 probabilities.

49 This coupling between inference and a fixed model set makes iterative model development
50 computationally inefficient.

51 Software design

52 `ABCdeZ.jl` is an open-source, MIT-licensed software package for Approximate Bayesian
53 Computation written in Julia (Bezanson et al., 2017) and hosted on [GitHub](#). It is registered
54 in the Julia General Registry and installable via `] add ABCdeZ`. The package provides a user-
55 friendly API, comprehensive documentation and minimal working examples for rapid onboarding.
56 For example, the code generating [Figure 1](#) is included in the documentation and repository.
57 `ABCdeZ.jl` uses automated continuous integration (CI) workflows for testing, documentation
58 deployment, and test coverage reporting. The test suite (`] test ABCdeZ`) achieves 98%
59 coverage (based on Codecov) within the CI workflow. The package depends only on widely
60 used and established Julia libraries `Random`, `Distributions`, `StatsBase` and `FLoops`, keeping
61 external dependencies minimal to ensure ease of installation and long-term maintainability.

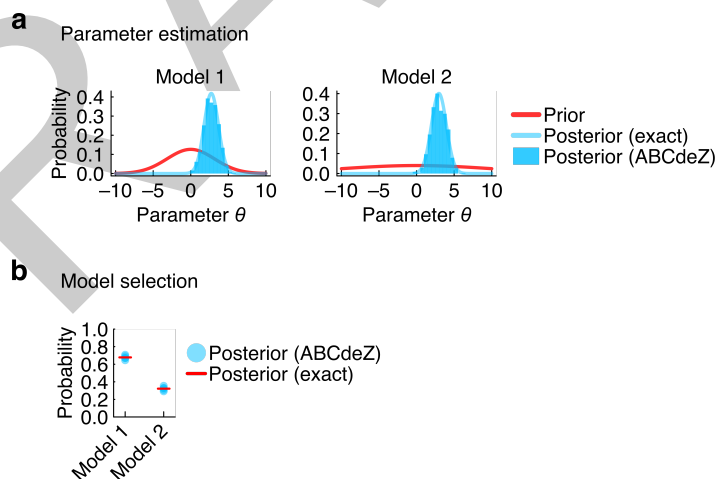


Figure 1: Minimal example from the `ABCdeZ.jl` documentation showcasing parameter inference and model comparison. Two models were independently fitted to a dataset, updating posterior parameter distributions from their priors (a). The estimated model evidences were subsequently used to compute posterior model probabilities from an initially uniform model prior (b). Inference results obtained with `ABCdeZ.jl` are compared with the exact analytical distributions, which can be derived for this minimal example but are generally unavailable in realistic research applications.

62 The core idea of `ABCdeZ.jl` is to infer model evidence estimates per model, independent of the
63 overall model set. From these model evidences, posterior model probabilities and Bayes factors
64 can then be computed for arbitrary subsets of analyzed models, allowing new models to be
65 added without recomputing previous inferences.

To enable the estimation of model evidences, ABCdeZ.jl implements an ABC-SMC framework in which particle weights are tracked (Del Moral et al., 2012; Didelot et al., 2011), following weight assignments analogous to those used in likelihood-based SMC inference (e.g., Amaya et al. (2021)). As SMC algorithms handle complex multimodal parameter landscapes well (Del Moral et al., 2006; Neal, 2001), high-quality posterior parameter samples are obtained alongside the evidence estimation. The algorithm uses differential evolution (Braak, 2006) for parameter proposals and stratified resampling (Douc et al., 2005) to maintain particle diversity.

To enforce the sequential progress of the sampling distribution towards the posterior, ABCdeZ.jl supports flexible distance constraint kernel definitions, including an indicator kernel (default) for strict stepwise progress and continuous distance kernels for softened progress constraints. Lightweight data blobs enable attachment of auxiliary information to particles throughout inference, including simulation outputs, simulation-to-data distances, or other metadata. Efficiency is a core design goal, and care was taken to avoid unnecessary memory allocations within the ABC-SMC implementation. For the minimal example included in the documentation, inference completes in approximately 0.03 seconds, increasing to roughly 0.1 seconds for a ten-dimensional parameter prior. These results indicate that, for realistic applications, the computational cost is typically dominated by the user-defined model simulation and distance evaluation rather than by the ABCdeZ.jl framework itself. Thread-safe parallelization via FLoops enables efficient multi-core execution while accommodating fast in-place mutable operations for distance evaluations.

Effective application of ABC methods requires careful design of summary statistics and distance functions. ABCdeZ.jl therefore provides extensive documentation covering practical aspects of simulation-based inference workflows. In addition, the package emphasizes ease of use through a simple API centered around a single top-level inference function (abcsdesmc!) and minimal working examples enabling straightforward adoption and reproducible workflows.

Research impact statement

ABCdeZ.jl was developed by the authors to address methodological demands arising from interdisciplinary research in systems biology. The software contributed substantially to two recent research studies (Frank et al., 2024; Ikeda et al., 2025), currently under review for publication (as of May 2026). The studies exemplify applications of ABCdeZ.jl to infer lineage pathways during tissue development in mammals, where large numbers of distinct cell types and functional tissues emerge from common progenitor cell types. Resolving such lineage pathways requires systematic and scalable model comparison, since the number of possible lineage topologies grows combinatorially. Figure 2 illustrates such a workflow for one lineage topology with three terminal cell types (A, B, C). In Frank et al. (2024), a systematic investigation of 86 distinct models was performed, representing a scale of model comparison that would be computationally demanding with ABC workflows requiring joint inference across all candidate models. By enabling per-model evidence estimation, ABCdeZ.jl supported iterative and extensible hypothesis testing in this large model space.

As increasingly complex biological datasets become available, we anticipate a growing demand for statistical inference tools that support scalable model comparison for simulation-based Bayesian inference. While motivated by systems biology applications, ABCdeZ.jl is a general-purpose framework that may prove useful in other research domains relying on likelihood-free inference.

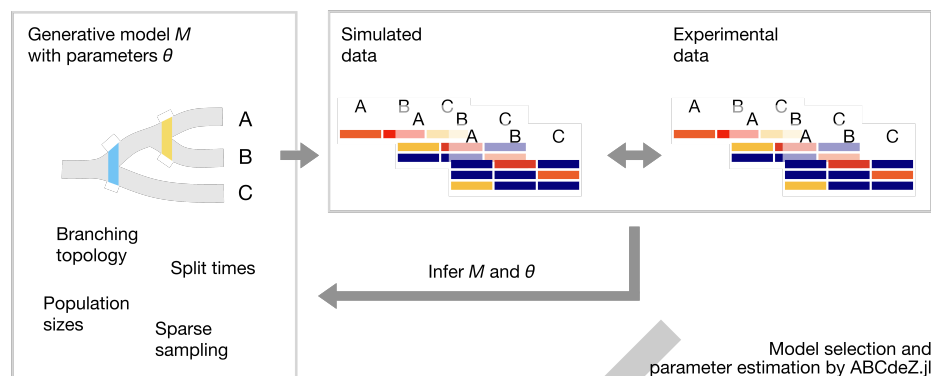


Figure 2: Representative workflow of ABCdeZ.jl adapted from a recent research application (Frank et al., 2024). ABCdeZ.jl enables inference of mechanistic processes underlying complex experimental data by combining generative forward simulations with large-scale and systematic model comparison. The framework estimates posterior parameter distributions and model evidences, from which posterior model probabilities can be derived to identify the most plausible explanatory processes underlying the observed data.

AI usage disclosure

Generative AI tools were used to improve grammar, readability and phrasing of the manuscript. No generative AI tools were used in the development of the software, the writing of this manuscript, or the preparation of supporting materials.

Acknowledgements

We thank Francesco Alemanno, author of the package [KissABC.jl](#), now publicly archived, for helpful correspondence and for the development of that package, parts of whose code base were used as a starting point for ABCdeZ.jl. In accordance with the original licensing terms, the corresponding shared license file is included in ABCdeZ.jl.

We thank all members of the Division of Theoretical Systems Biology at the German Cancer Research Center for testing and using ABCdeZ.jl, thereby providing valuable feedback for previous releases of ABCdeZ.jl.

References

- Amaya, M., Linde, N., & Laloy, E. (2021). Adaptive sequential monte carlo for posterior inference and model selection among complex geological priors. *Geophysical Journal International*, 226(2), 1220–1238. <https://doi.org/10.1093/gji/ggab170>
- Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1), 65–98. <https://doi.org/10.1137/141000671>
- Braak, C. J. F. T. (2006). A markov chain monte carlo version of the genetic algorithm differential evolution: Easy bayesian computing for real parameter spaces. *Statistics and Computing*, 16(3), 239–249. <https://doi.org/10.1007/s11222-006-8769-1>
- Del Moral, P., Doucet, A., & Jasra, A. (2006). Sequential monte carlo samplers. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 68(3), 411–436. <https://doi.org/10.1111/j.1467-9868.2006.00553.x>
- Del Moral, P., Doucet, A., & Jasra, A. (2012). An adaptive sequential monte carlo method for approximate bayesian computation. *Statistics and Computing*, 22(5), 1009–1020.

- 136 <https://doi.org/10.1007/s11222-011-9271-y>
- 137 Didelot, X., Everitt, R. G., Johansen, A. M., & Lawson, D. J. (2011). Likelihood-free estimation
138 of model evidence. *Bayesian Analysis*, 6(1). <https://doi.org/10.1214/11-BA602>
- 139 Douc, R., Cappé, O., & Moulines, E. (2005). *Comparison of resampling schemes for particle*
140 *filtering*. <https://doi.org/10.48550/ARXIV.CS/0507025>
- 141 Frank, L., Postrach, D., Langhinrichs, M., Höfer, T., & Rodewald, H.-R. (2024). *Holistic*
142 *genetic barcoding reveals a lineage tree of tissue macrophage development*. bioRxiv.
143 <https://doi.org/10.1101/2024.11.29.625985>
- 144 Ikeda, T., Langhinrichs, M., Nizharadze, T., Koike, C., Kato, Y., Yamaguchi, K., Shigenobu,
145 S., Yoshido, K., Suzuki, S., Nakagawa, T., Maruyama, A., Mizuno, S., Takahashi, S.,
146 Becker, N. B., Rodewald, H.-R., Höfer, T., & Yoshida, S. (2025). *Early development of*
147 *male germ cell clones shapes their reproductive success*. bioRxiv. [https://doi.org/10.1101/](https://doi.org/10.1101/2025.09.02.672339)
148 [2025.09.02.672339](https://doi.org/10.1101/2025.09.02.672339)
- 149 Neal, R. M. (2001). Annealed importance sampling. *Statistics and Computing*, 11(2), 125–139.
150 <https://doi.org/10.1023/A:1008923215028>
- 151 Schälte, Y., Klinger, E., Alamoudi, E., & Hasenauer, J. (2022). pyABC: Efficient and robust
152 easy-to-use approximate bayesian computation. *Journal of Open Source Software*, 7(74),
153 4304. <https://doi.org/10.21105/joss.04304>
- 154 Tankhilevich, E., Ish-Horowicz, J., Hameed, T., Roesch, E., Kleijn, I., Stumpf, M. P. H.,
155 & He, F. (2020). GpABC: A julia package for approximate bayesian computation with
156 gaussian process emulation. *Bioinformatics*, 36(10), 3286–3287. [https://doi.org/10.1093/](https://doi.org/10.1093/bioinformatics/btaa078)
157 [bioinformatics/btaa078](https://doi.org/10.1093/bioinformatics/btaa078)